AutoRegressivo – AR

Daniel Freitas

Instituto J&F
Centro de Educação para os Negócios

Modelo AutoRegressivo - AR

Modelo AutoRegressivo: definição

O modelo **autoregressivo (AR)** é um modelo de séries temporais em que o valor atual da série depende linearmente de valores passados da própria série.

$$Y_t = \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \cdots + \phi_p Y_{t-p} + \varepsilon_t$$

Y_t = valor atual da série

$\phi_1, \phi_2, \ldots, \phi_p$ = coeficientes autorregressivos

p = ordem do modelo (número de lags considerados)

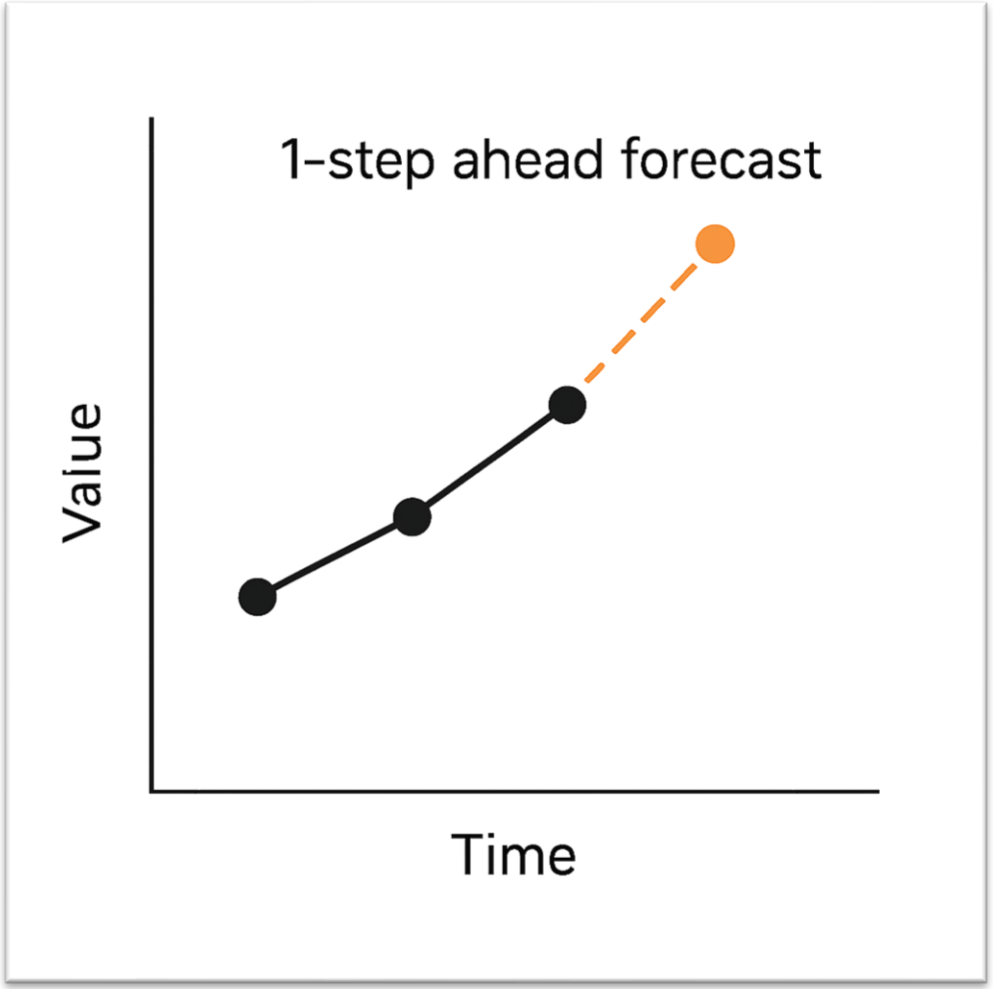
ε_t = erro aleatório (ruído branco)

AR: quando utilizar ?

- Quando a série é estacionária ou foi transformada para ser;
- Quando os valores passados tem relação com os dados futuros;
- Quando se deseja algo mais sofisticado do que os Base Models;

AR: como funciona ?

- **.fit()** – ajusta os pesos dos coeficientes
- **.predict()** - utiliza os pesos aprendidos para realizar a previsão do próximo valor



Série Temporal

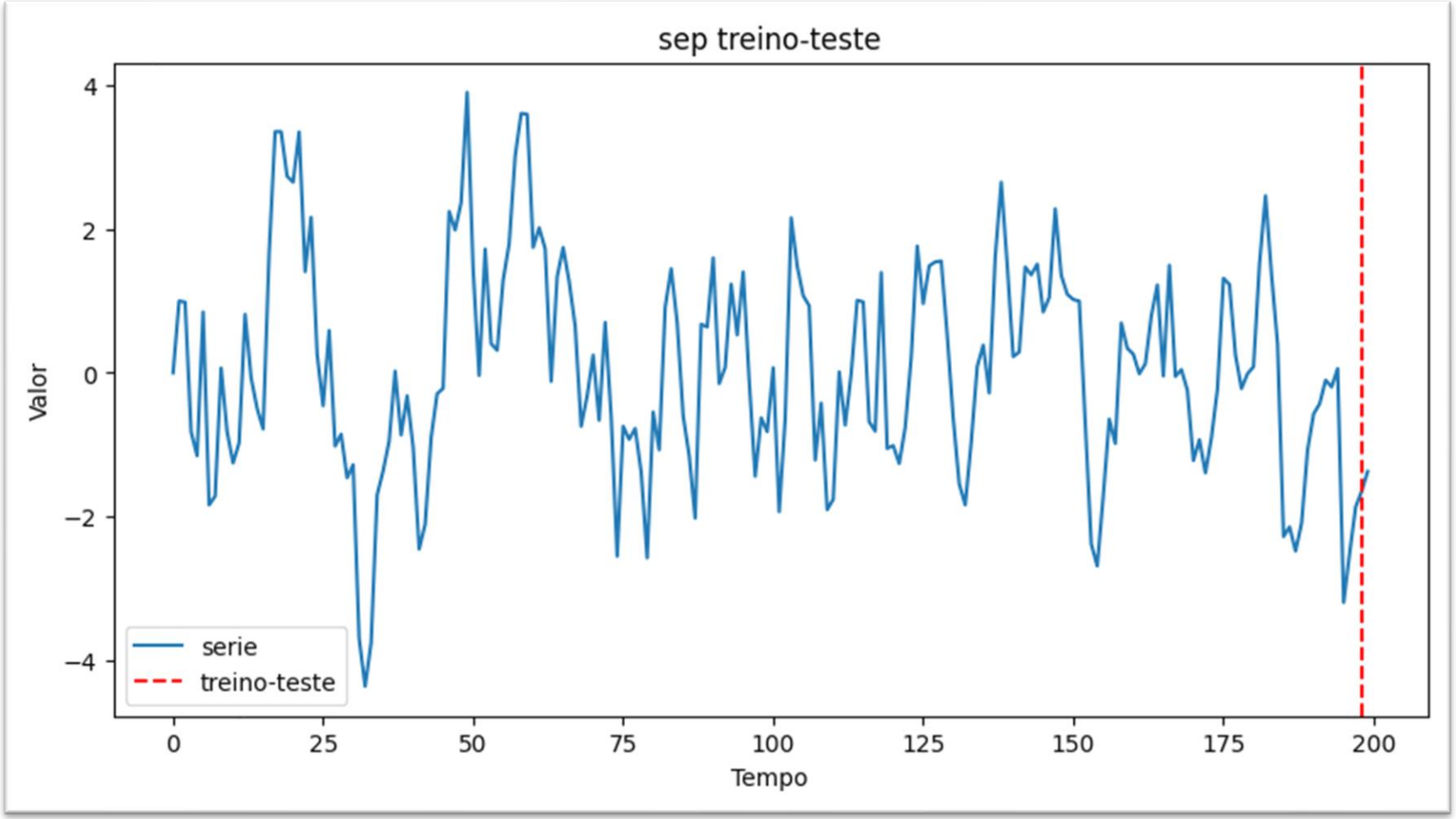
Série:

Considere a série temporal serie_aula_ar.xlsx para prevermos seu valores. Baixe o arquivo, e implemente o seu código. Realize as importações abaixo:

```
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.graphics.tsaplots import plot_pacf
from statsmodels.tsa.stattools import kpss, adfuller
from statsmodels.tsa.ar_model import AutoReg
from sklearn.metrics import mean_absolute_error
```

Treino e teste:

Realize a plotagem da série e sua separação treino e teste. Considere 99 valores par ao treino e 1 para o teste:



Treino e teste:

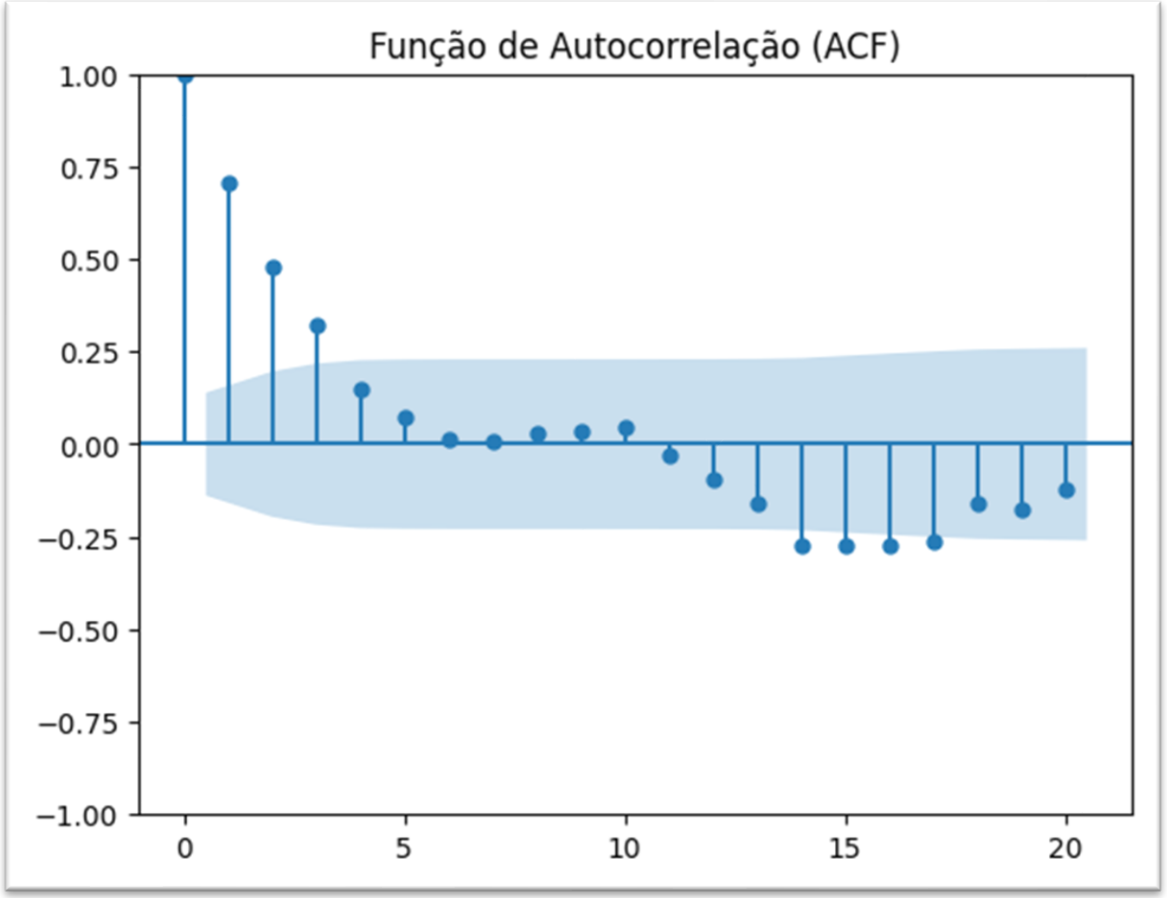
Realize a plotagem da série e sua separação treino e teste. Considere 99 valores par ao treino e 1 para o teste:

```
# treino e teste
serie = serie_original[:-1]   # 99 - treino
teste  = serie_original[-1:]   # 1 - teste


# pltando a serie
plt.figure(figsize=(10, 5))
plt.plot(serie_original, label="serie")
plt.axvline(x=len(serie)-1, color="red", linestyle="--", label="treino-teste")
plt.xlabel("Tempo")
plt.ylabel("Valor")
plt.title("sep treino-teste")
plt.legend()
plt.show()
```

Estacionariedade

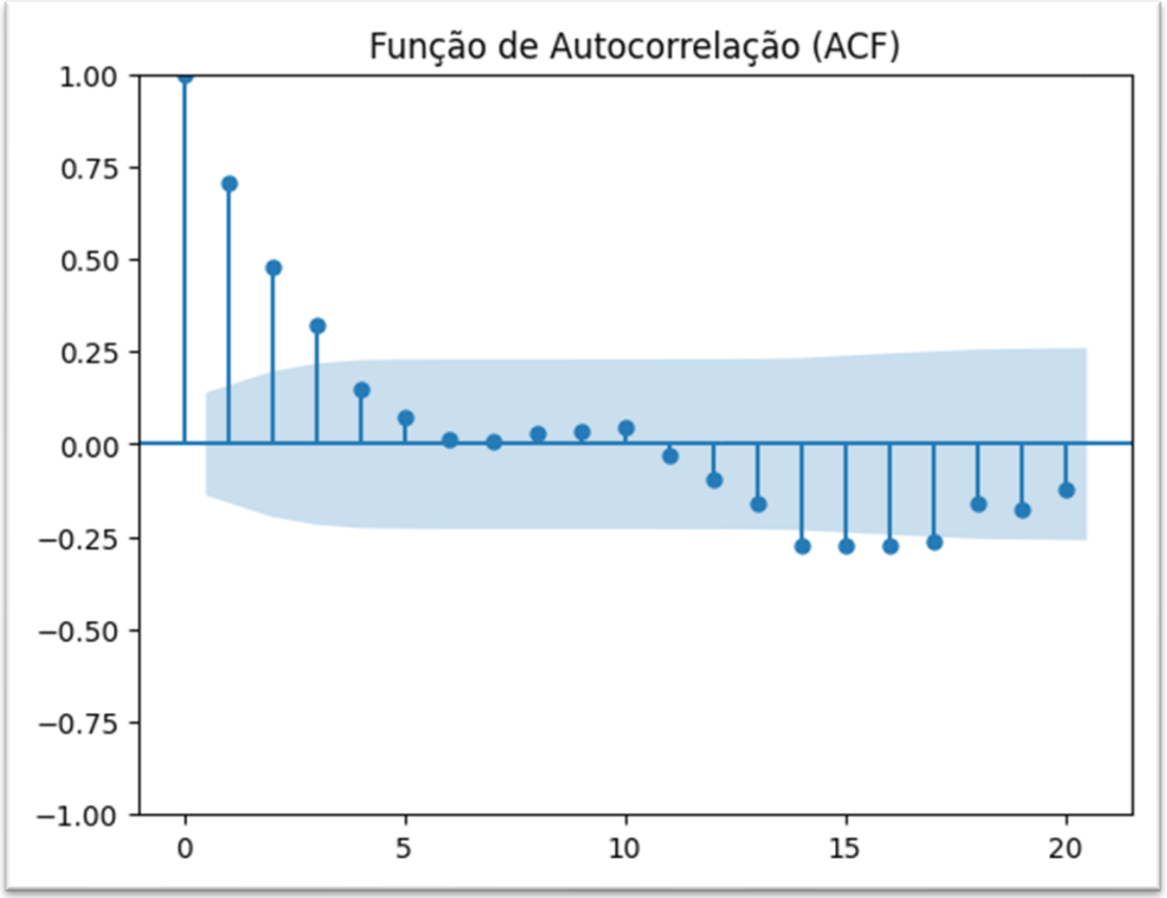
Estacionariedade:

Faça a plotagem do gráfico ACF e realize os testes de estacionariedade ADF e KPSS:



Estacionariedade:

Faça a plotagem do gráfico ACF e realize os testes de estacionariedade ADF e KPSS:




```
KPSS
p-valor: 0.1

ADF
 p-valor: 7.07928686589191e-07
```

Estacionariedade:

Faça a plotagem do gráfico ACF e realize os testes de estacionariedade ADF e KPSS:

```
# ACF
plt.figure(figsize=(8, 4))
plot_acf(serie, lags=20)
plt.title("Função de Autocorrelação (ACF)")
plt.show()

# ADF e KPSS
kpss_p = kpss(serie)[1]
adf_p= adfuller(serie)[1]

print("KPSS")
print(f"p-valor: {kpss_p}")
print()

print("ADF")
print(f" p-valor: {adf_p}")
```

PACF – breve explicação

PACF: Partial Autocorrelation Function

- Mede a correlação entre a série e seus valores defasados removendo o efeito intermediário dos lags menores.
- Foca no impacto “puro” de cada lag, sem interferência dos anteriores.
- Serve principalmente para identificar até qual lag um modelo AR deve considerar.
- No gráfico, o último lag significativo antes de cair para a zona de confiança indica o limite máximo para p.
- No ACF, o valor de um lag pode ser alto não porque esse lag explica diretamente o presente, mas porque ele está fortemente relacionado a lags anteriores que explicam o presente
- O que acontece é que essa “relação indireta” não está escrita na fórmula, mas sim **embutida nos valores da série sobre os quais a fórmula atua**.

PACF: formulação matemática

Equações de Yule–Walker recursivas:

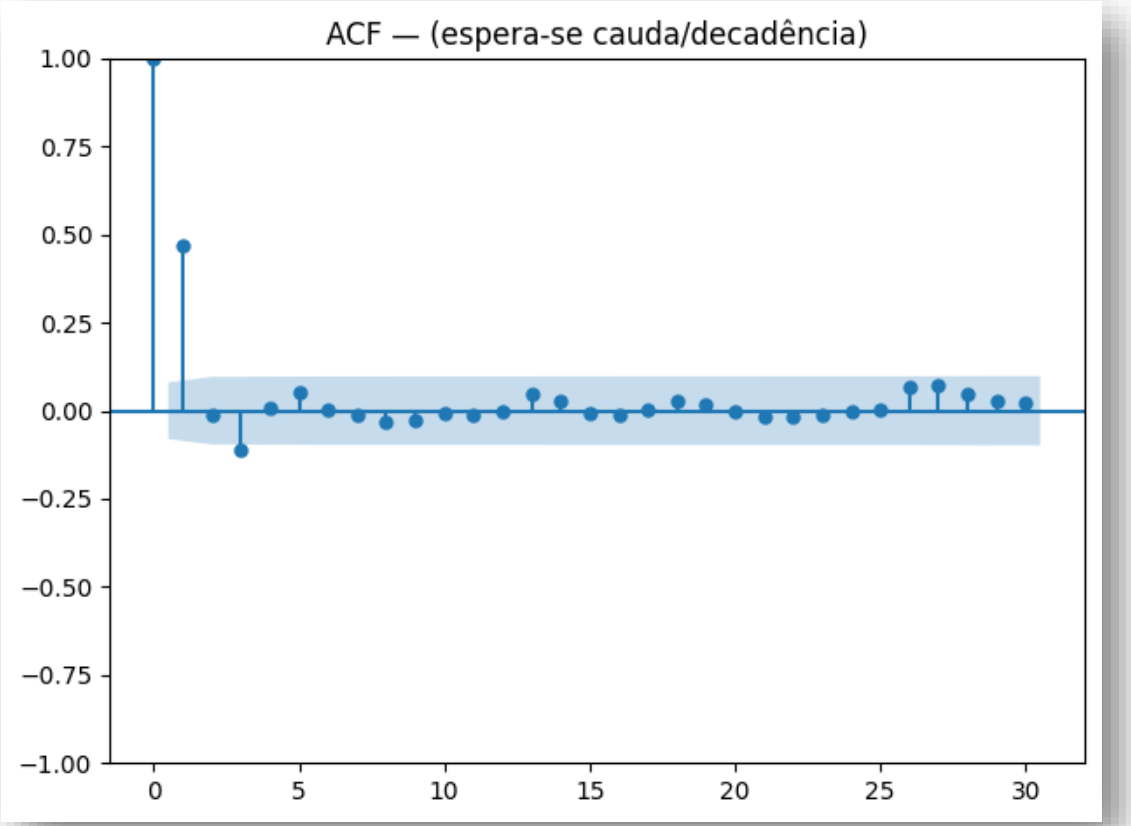
k = 1

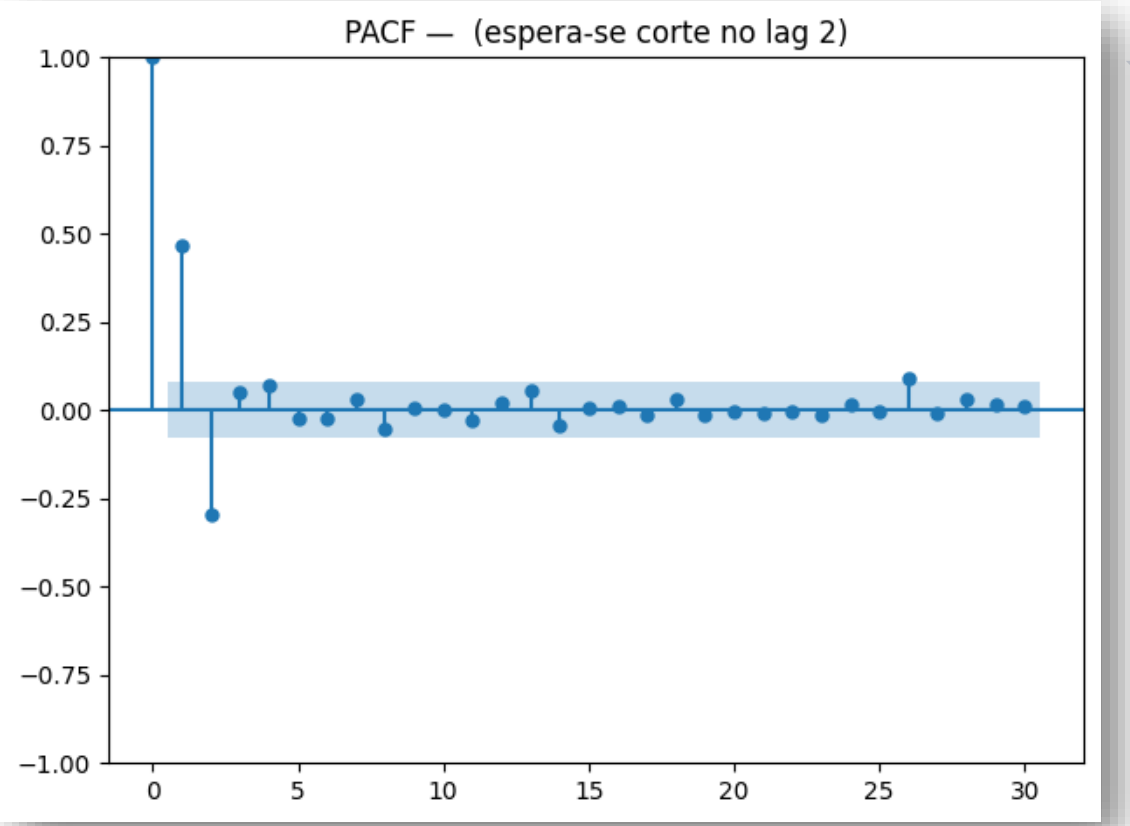
$$\phi_{1,1} = \rho(1)$$

k > 1

$$\phi_{k,k} = \frac{\rho(k) - \sum_{j=1}^{k-1} \phi_{k-1,j}\rho(k-j)}{1 - \sum_{j=1}^{k-1} \phi_{k-1,j}\rho(j)}$$

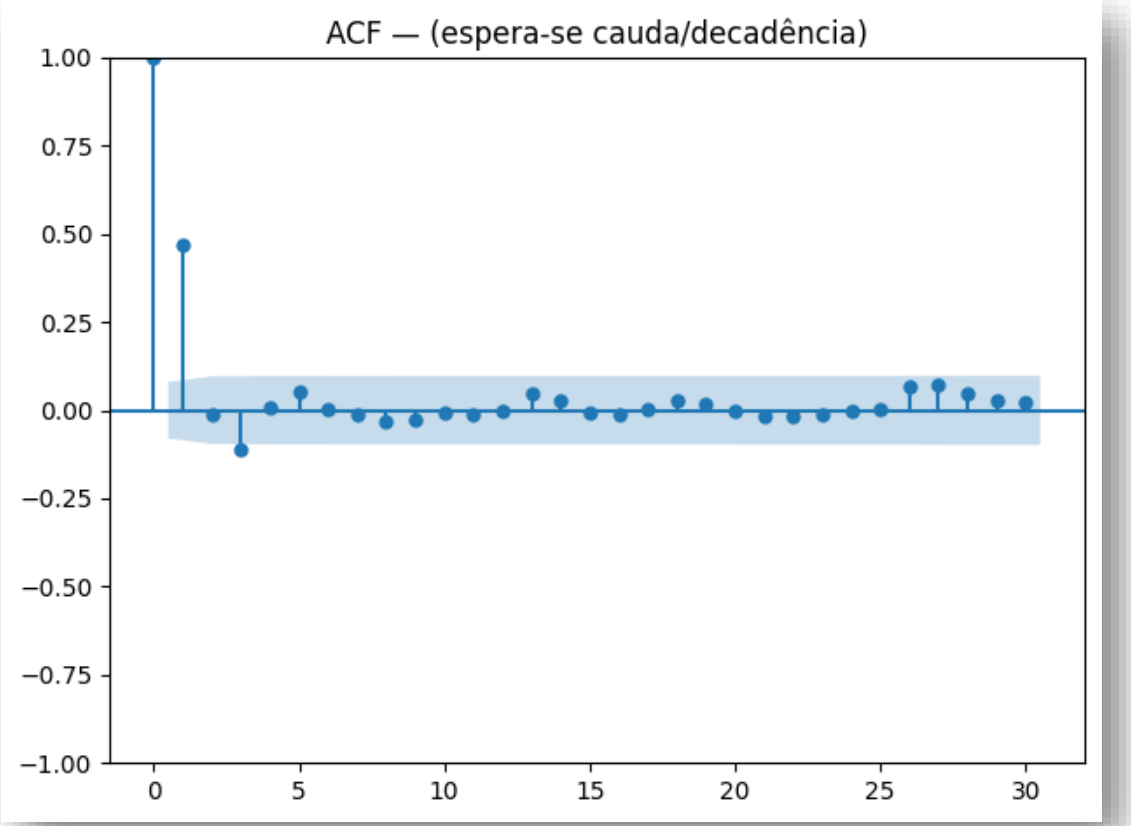
ACF vs PACF:

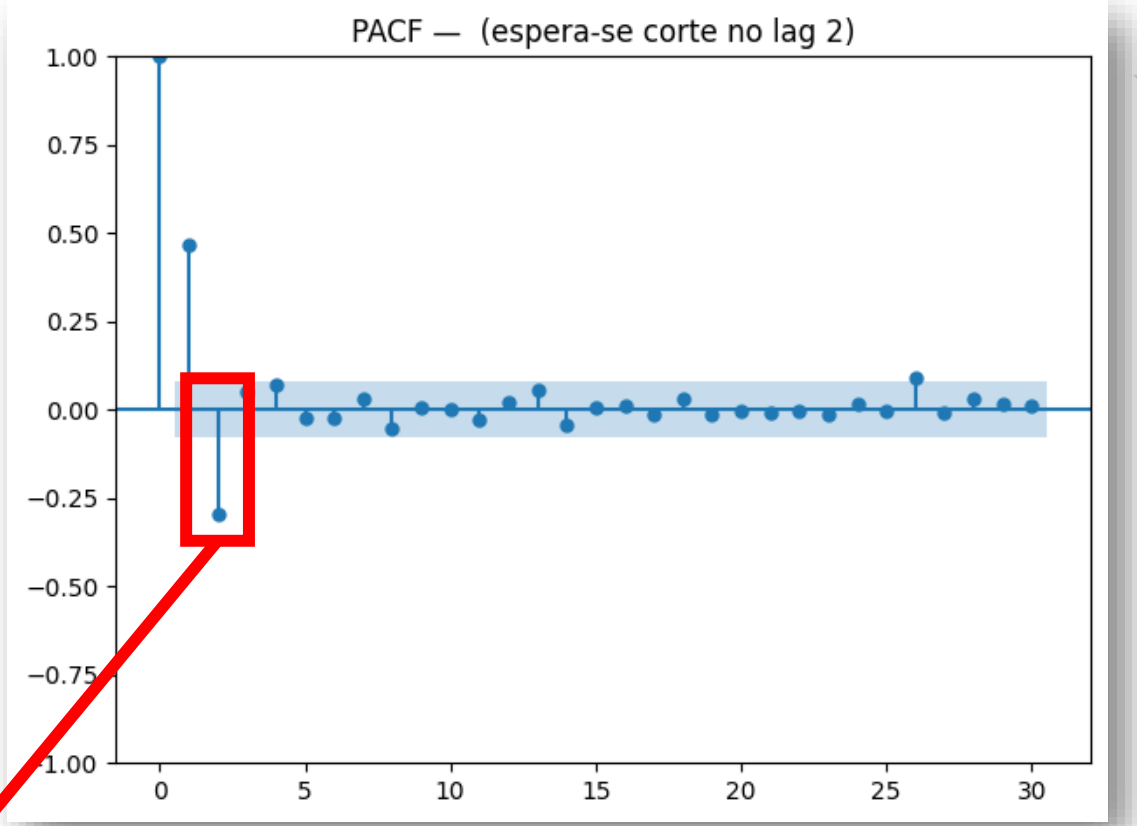





```
from statsmodels.graphics.tsaplots import plot_pacf
```

ACF vs PACF:



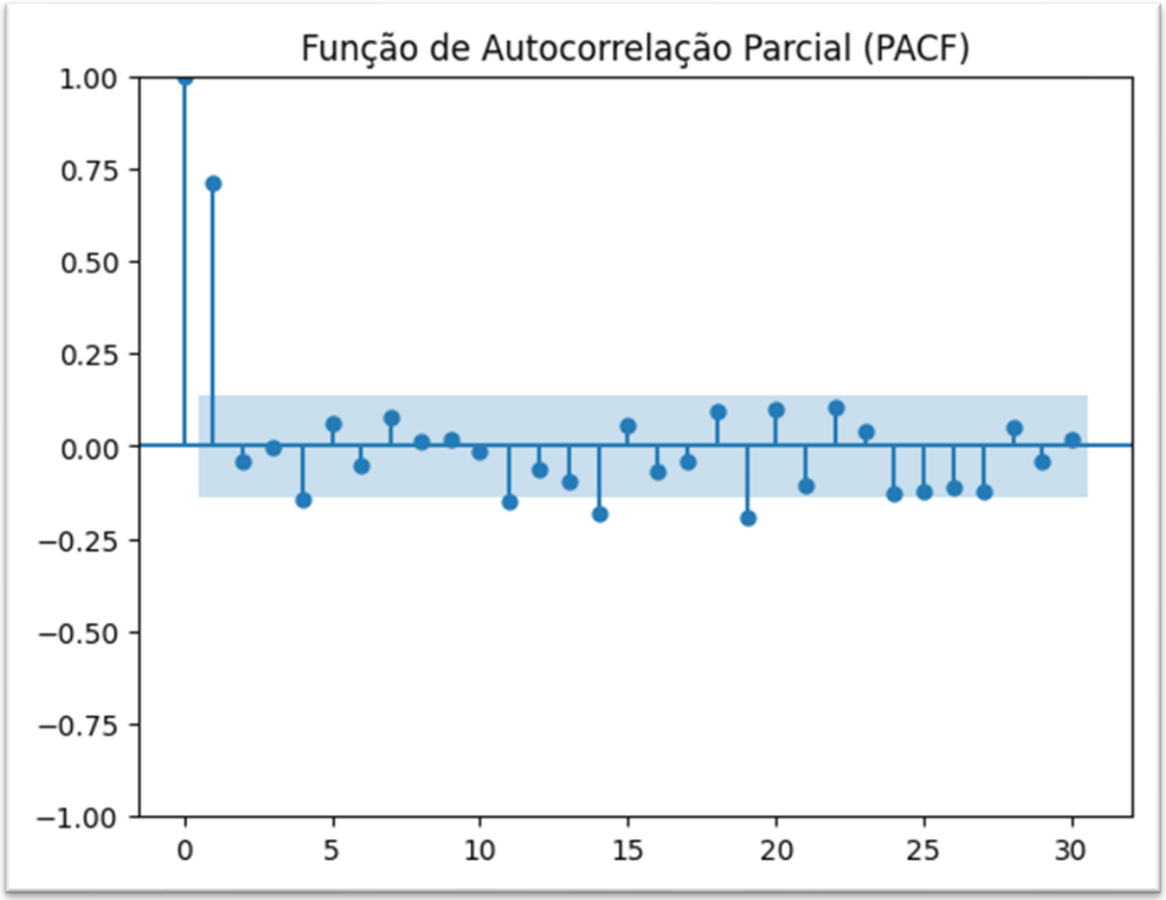


Pode indicar que o lag = 2 pode ser um bom lag para o AR(p) nessa série de exemplo

PACF – para nossa serie

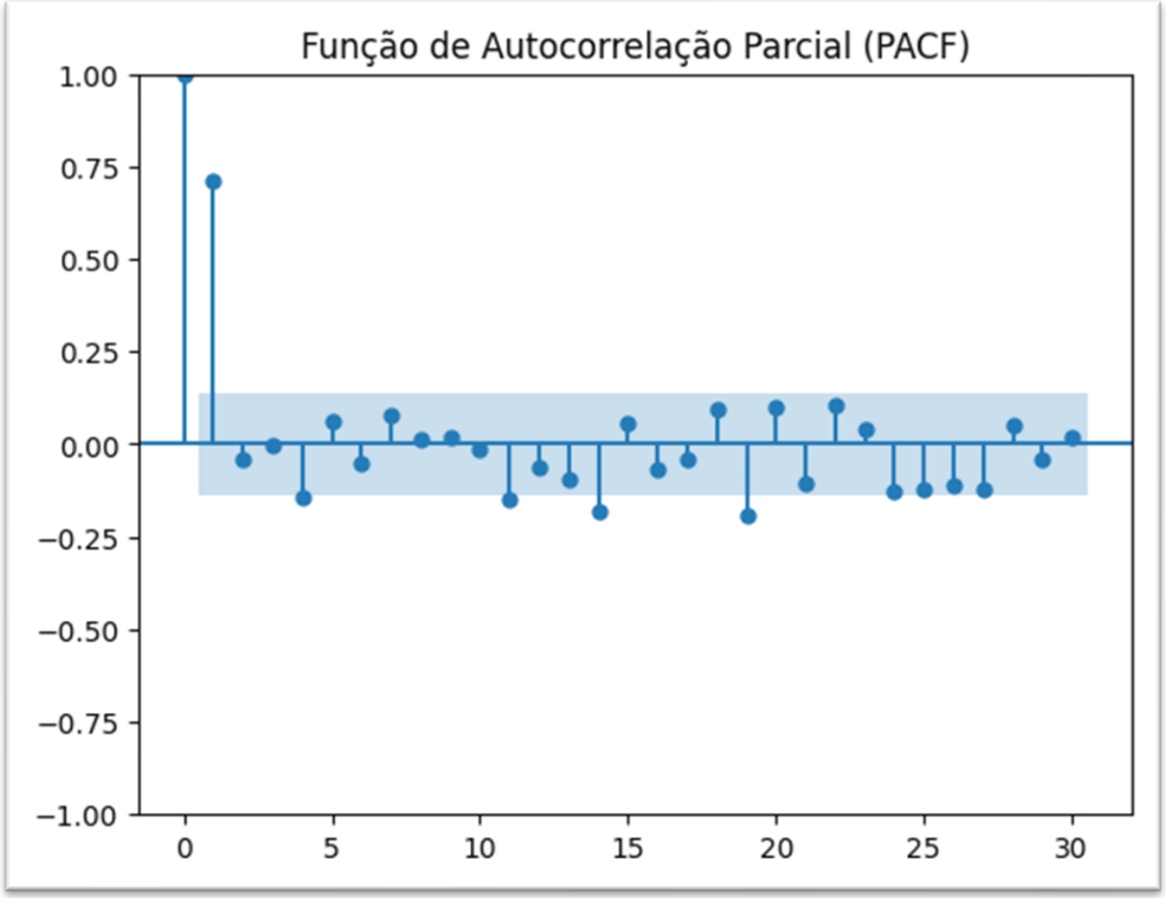
PACF: Partial Autocorrelation Function

Realize a plotagem do PACF para nossa série e escolha o p adequado para utilizarmos no modelo autoregressivo:



PACF: Partial Autocorrelation Function

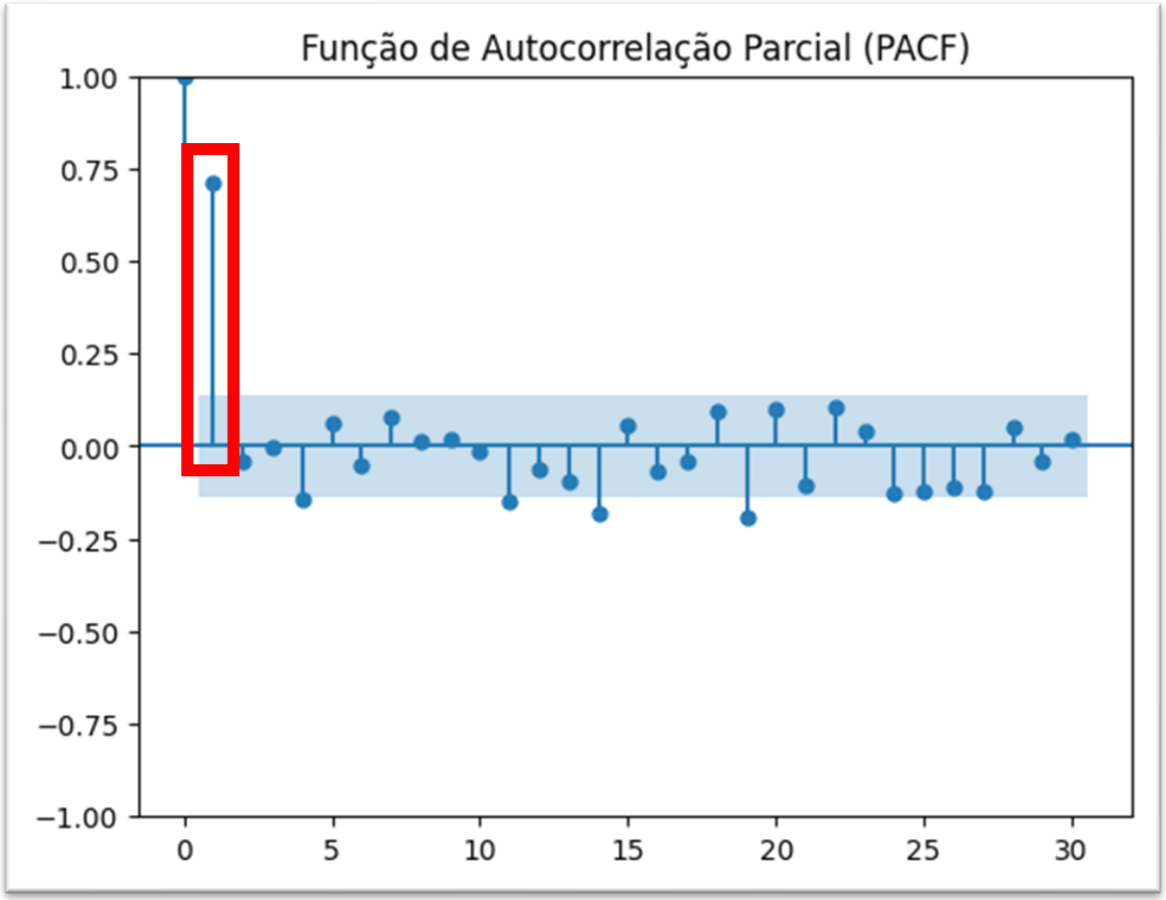
Realize a plotagem do PACF para nossa série e escolha o p adequado para utilizarmos no modelo autoregressivo:




```
plt.figure(figsize=(8, 4))
plot_pacf(serie, lags=30)
plt.title("Função de Autocorrelação Parcial (PACF)")
plt.show()
```

PACF: Partial Autocorrelation Function

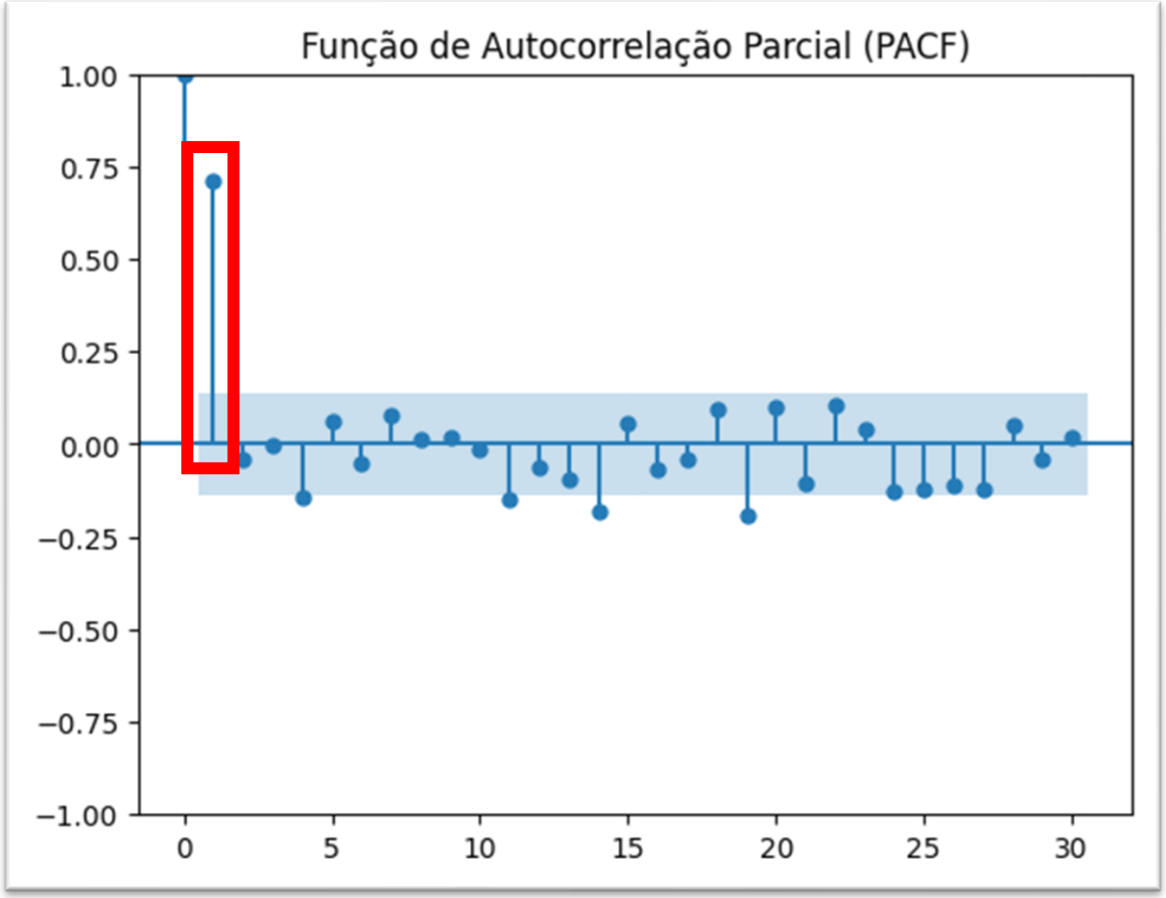
Realize a plotagem do PACF para nossa série e escolha o p adequado para utilizarmos no modelo autoregressivo:




```
plt.figure(figsize=(8, 4))
plot_pacf(serie, lags=30)
plt.title("Função de Autocorrelação Parcial (PACF)")
plt.show()
```

PACF: Partial Autocorrelation Function

Realize a plotagem do PACF para nossa série e escolha o p adequado para utilizarmos no modelo autoregressivo:




```
plt.figure(figsize=(8, 4))
plot_pacf(serie, lags=30)
plt.title("Função de Autocorrelação Parcial (PACF)")
plt.show()
```

Será que não existe um método analítico de escolha do p ? 🤔

AR – aplicando o modelo

AR(p): aplicando na prática

Agora vamos aplicar o modelo na prática com o p escolhido na forma gráfica pelo PACF:

```
#  criacao do melhor modelo e previsão do valor real
modelo_final = AutoReg(serie, lags=1, old_names=False).fit()
previsao = modelo_final.predict(start=len(serie), end=len(serie))[0]
mae_teste = mean_absolute_error(teste, [previsao])

print('Previsao:',previsao)
print('Valor real:',teste[0])
print('MAE:',mae_teste)
print('MAPE %:',abs(mae_teste/teste[0])*100,'%')
```

```
Previsao: -1.1596179034538698
Valor real: -1.3693876501661622
MAE: 0.20976974671229232
MAPE %: 15.318507267597948 %
```

BIC

BIC: Bayesian Information Criterion

- Também chamado critério de Schwarz, é um critério de seleção de modelos que **equilibra ajuste e simplicidade**;
- **Menor BIC é melhor**. O primeiro termo recompensa bom ajuste; o segundo pune modelos com muitos parâmetros (complexos);
- Devemos testar vários p e decidir pelo modelo que apresenta o menor BIC

$$\mathrm{BIC} = -2\ln L(\hat{\theta}) + k\ln(n_{\mathrm{eff}})$$

$L(\hat{\theta})$ = verossimilhança no ótimo (quanto maior, melhor o ajuste),
k = número de parâmetros estimados,
n_{eff} = número de observações efetivas usadas no ajuste (em AR(p), costuma ser $n - p$ por causa dos lags iniciais).

BIC: escolhendo o modelo

- Pelo método BIC testamos vários p;
- Testamos vários p, mas não colocamos p muito grandes, eles ajustam demais no modelo e tendem a errar;

```
p_max = 20  # testando alguns lags (nao coloque mto alto

# inicializando variaveis
melhor_p = None
melhor_modelo = None
menor_bic = np.inf # infinito pois qualquer bic incial ganhha

for p in range(1, p_max+1):
    m = AutoReg(serie, lags=p, old_names=False).fit()
    if m.bic < menor_bic:
        # guardando melhor modelo e suasss infos
        melhor_p = p
        melhor_modelo = m
        menor_bic = m.bic
        print(menor_bic)

print("Melhor p:",melhor_p)
print("Menor BIC:",menor_bic)
```

BIC: escolhendo o modelo

- Pelo método BIC testamos vários p;
- Testamos vários p, mas não colocamos p muito grandes, eles ajustam demais no modelo e tendem a errar;

```
Melhor p: 1
Menor BIC: 598.8103471970958
```

```python
p_max = 20  # testando alguns lags (nao coloque mto alto

# inicializando variaveis
melhor_p = None
melhor_modelo = None
menor_bic = np.inf # infinito pois qualquer bic incial ganhha

for p in range(1, p_max+1):
    m = AutoReg(serie, lags=p, old_names=False).fit()
    if m.bic < menor_bic:
        # guardando melhor modelo e suasss infos
        melhor_p = p
        melhor_modelo = m
        menor_bic = m.bic
        print(menor_bic)

print("Melhor p:",melhor_p)
print("Menor BIC:",menor_bic)
```

BIC: escolhendo o modelo

- Pelo método BIC testamos vários p;
- Testamos vários p, mas não colocamos p muito grandes, eles ajustam demais no modelo e tendem a errar;

```
Melhor p: 1
Menor BIC: 598.8103471970958
```

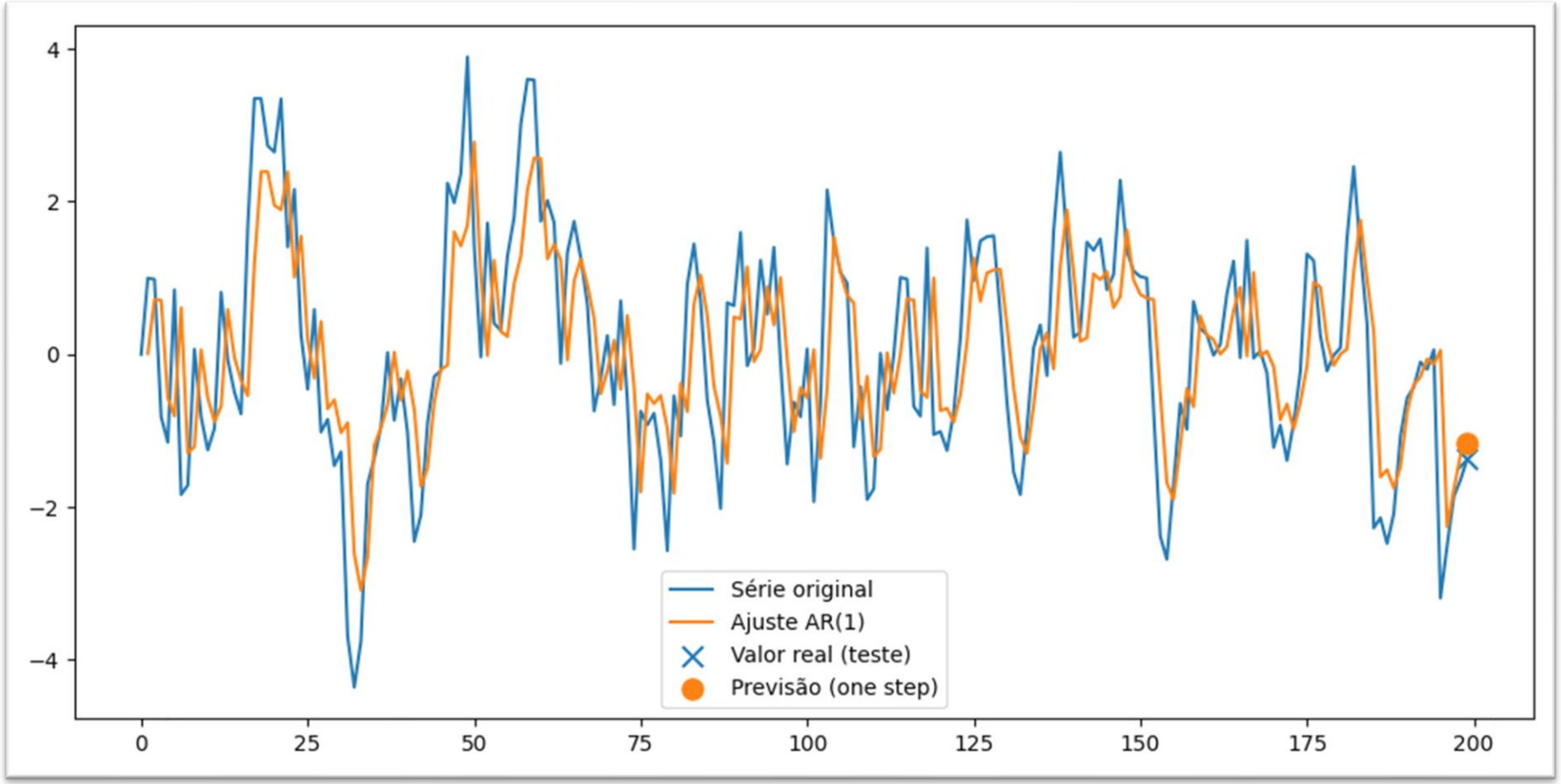
O método do PACF e o método do BIC concordaram! p=1 parece bom! 😎😎😎

```python
p_max = 20  # testando alguns lags (nao coloque mto alto

# inicializando variaveis
melhor_p = None
melhor_modelo = None
menor_bic = np.inf # infinito pois qualquer bic incial ganhha

for p in range(1, p_max+1):
    m = AutoReg(serie, lags=p, old_names=False).fit()
    if m.bic < menor_bic:
        # guardando melhor modelo e suasss infos
        melhor_p = p
        melhor_modelo = m
        menor_bic = m.bic
        print(menor_bic)

print("Melhor p:",melhor_p)
print("Menor BIC:",menor_bic)
```

Verificando ajuste do modelo

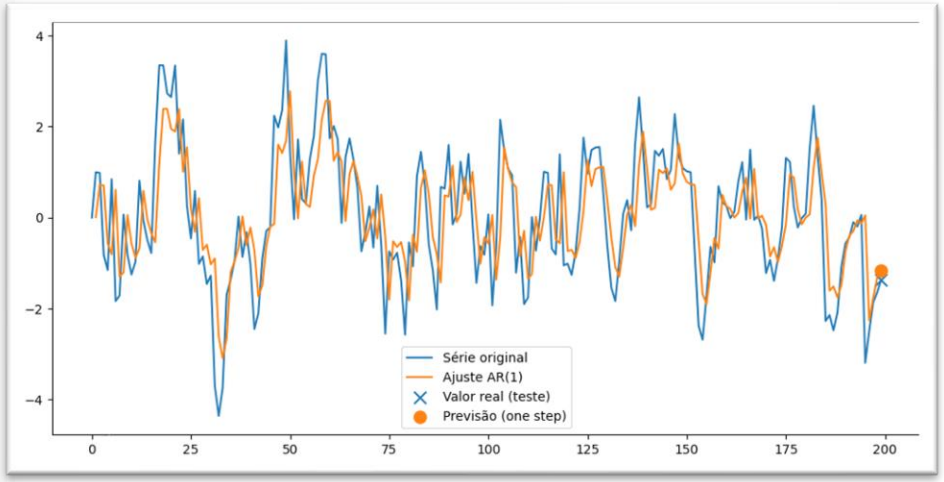
Sobrepondo séries:

Para verificar o ajuste do modelo vamos sobrepor as séries, considerando também os valores de previsão vs real:



Sobrepondo séries:

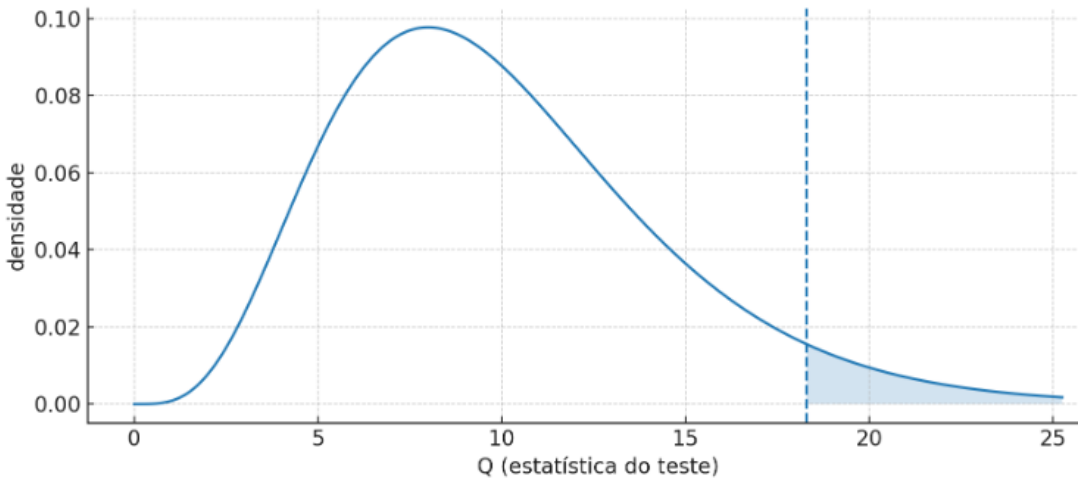
```python
# devemos comecar a partir do p pois perdemos p valores por conta do lag
p = 1  # o p vencedor


# ajuste dentro da amostra
serie_ajustada_modelo = modelo_final.predict(start=p, end=len(serie)-1)
x_fit = np.arange(p, len(serie))

# sobrepondo a serie original com o modelo ajustado + a psrevisao
plt.figure(figsize=(10,5))
line_orig, = plt.plot(serie_original, label='Série original')
line_fit,  = plt.plot(x_fit, serie_ajustada_modelo, label=f'Ajuste AR({p})')

# colocando o ponto previsto e o ponto real
plt.scatter([len(serie)], [teste[0]], s=90, marker='x',
            color=line_orig.get_color(), label='Valor real (teste)')
plt.scatter([len(serie)], [previsao], s=90, marker='o',
            color=line_fit.get_color(), label='Previsão (one step)')

plt.legend()
plt.tight_layout()
plt.show()
```



Autocorrelação nos resíduos

Ljung-Box:

- É um teste "**portmanteau**" (tudo em um) que verifica se os resíduos ainda têm autocorrelação conjunta até um lag h
- ele checa conjuntamente se todas as autocorrelações até o lag h são zero (resíduos = ruído), em vez de olhar lag por lag separadamente



$\boldsymbol{H_0}$: não há autocorrelação nos resíduos até h

$\boldsymbol{H_1}$: existe autocorrelação nos lags

Se:

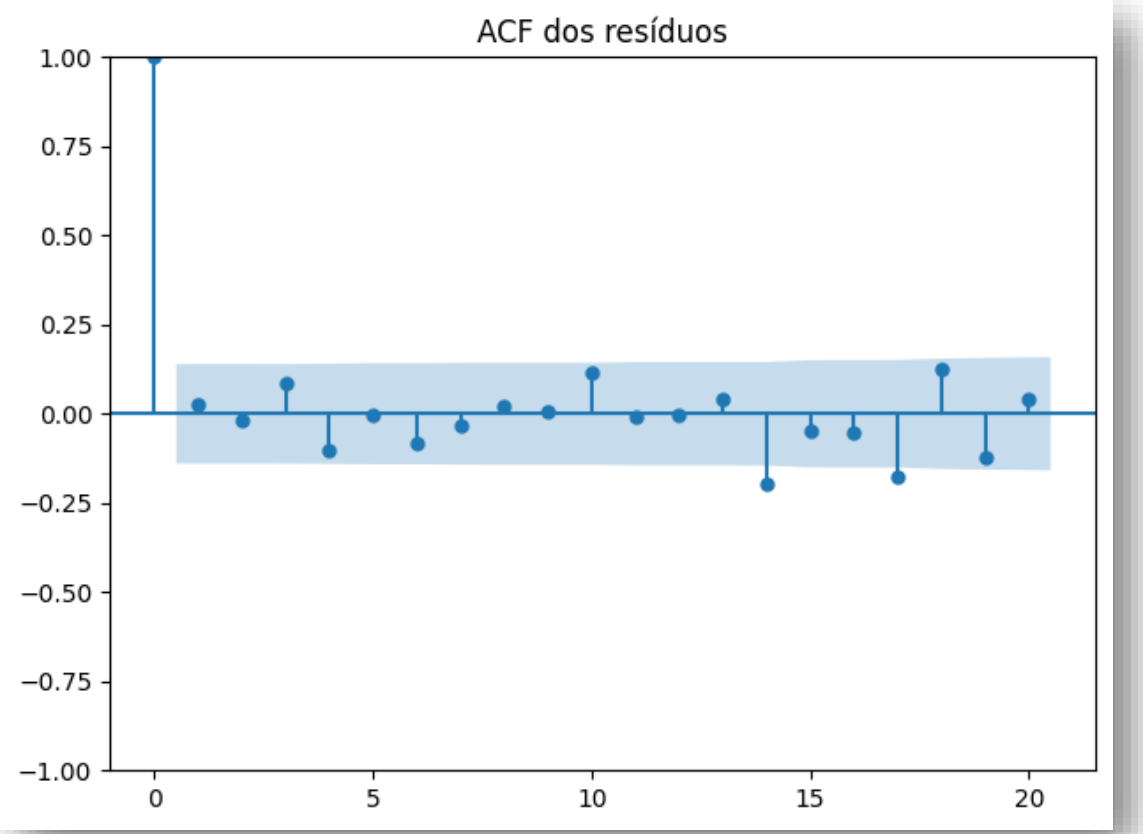
- $p \leq 0.05 \rightarrow$ rejeita H_0
- $p > 0.05 \rightarrow$ não rejeita H_0

Queremos NÃO rejeitar $\boldsymbol{H_0}$

Ljung-Box:

```python
from statsmodels.stats.diagnostic import acorr_ljungbox

# capturando residuos
res = modelo_final.resid

# Ljung-Box em lag 20 (pod ajustar)
lb = acorr_ljungbox(res, lags=[20])
print("Ljung Box (lag=20):")
print(lb)

# ACF dos residuos - esperamos valores nas bandas
plt.figure(figsize=(8,4))
plot_acf(res, lags=20)
plt.title('ACF dos resíduos')
plt.tight_layout()
plt.show()
```

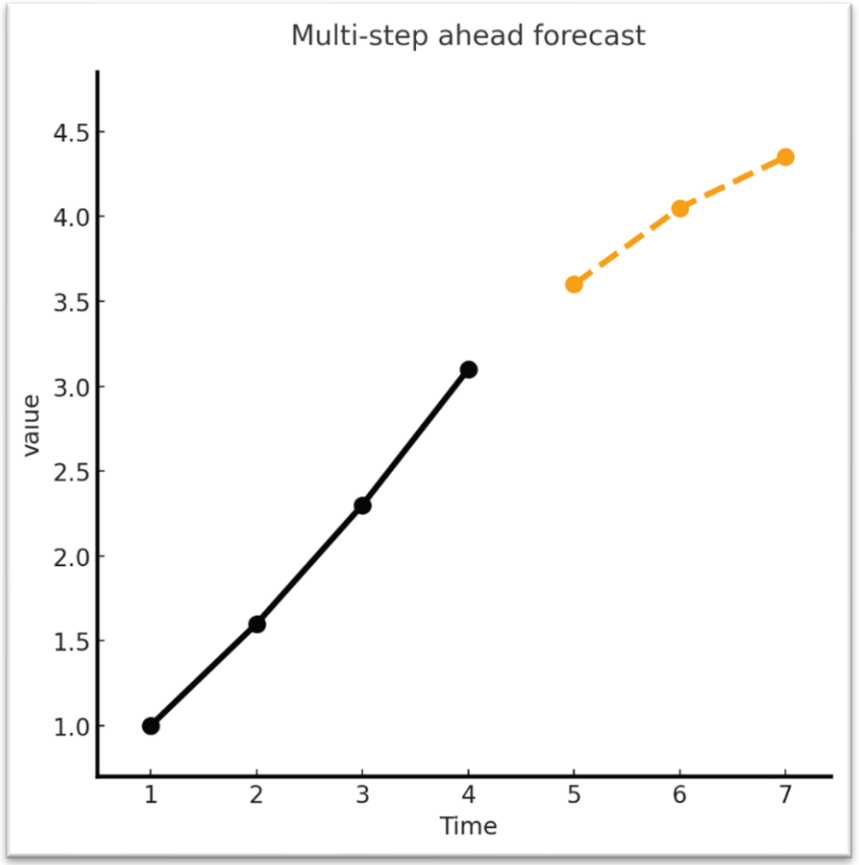
```
Ljung-Box (lag=20):
      lb_stat  lb_pvalue
20  31.980426   0.043507
```



multi-step-ahead (h-step)

Previsão "multi-step ahead":

- A previsão multi-step ahead (ou h-step ahead) é quando estimamos não só o próximo valor, mas uma **sequência de H valores no futuro**
- Na prática, prevemos t+1, depois t+2, … até t+H.
- É uma previsão de médio prazo, feita passo a passo, e a incerteza cresce a cada passo;
- **Devemos separar o treino e teste de uma forma maior**



Pipeline:

Realize a separação treino teste a seguir, vamos prever 4 valores no futuro! 🔮 🔮

```
# defina horizonte de teste (H passos à frente)

horizonte = 4
train = serie_original[:-horizonte]
test  = serie_original[-horizonte:]
```

Após isso, realize o mesmo pipeline (fluxo de trabalho):

1. ACF;
2. Testes ADF e KPSS;
3. PACF e BIC;
4. Escolha do melhor modelo;

Previsão e erro:

```python
#  previsao recursiva multi-step (t+1 ... t+horizonte)
previsoes = modelo_escolhido.predict(start=len(train), end=len(train)+horizonte-1)

# avaliacao por horizonte e meddia
abs_err = np.abs(previsoes - test)
df_eval = pd.DataFrame({
    'horizonte': np.arange(1, horizonte+1),
    'previsto': previsoes,
    'real': test,
    'abs_err': abs_err
})
print(df_eval)
print("MAE médio:", abs_err.mean())
```

```
Melhor p (BIC): 1 | BIC=592.744
   horizonte  previsto      real   abs_err
0          1 -2.241075 -2.502161  0.261086
1          2 -1.570482 -1.862364  0.291882
2          3 -1.096489 -1.644916  0.548428
3          4 -0.761457 -1.369388  0.607931
MAE médio: 0.42733164266300091
```

Gráfico e próximos passos

Realize a plotagem da série, contendo:

- Série original;
- Valores previstos;
- Valores reais

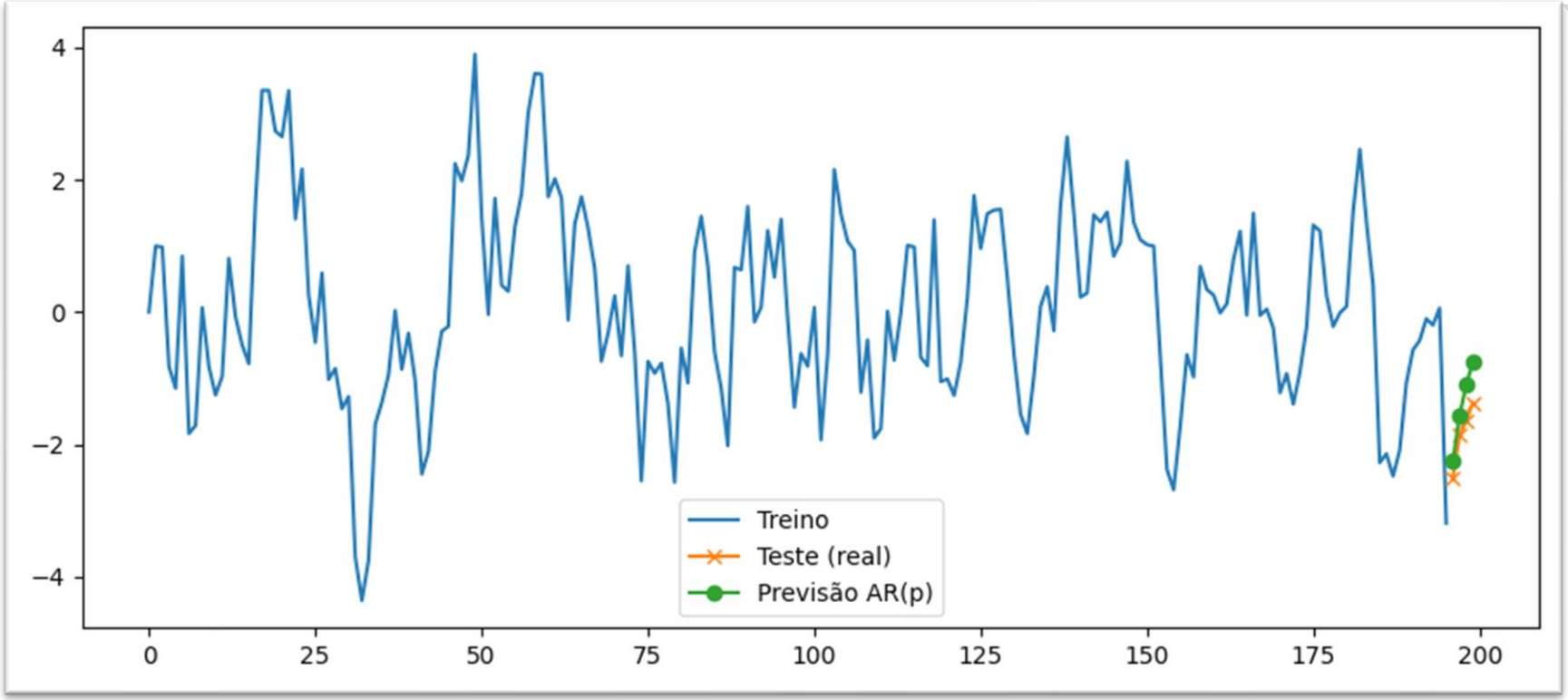


Gráfico e próximos passos

```
# plotando a curva originall + previsao + valores reais esperados
plt.figure(figsize=(9,4))
plt.plot(range(len(train)), train, label='Treino')
plt.plot(range(len(train), len(train)+horizonte), test, marker='x', label='Teste (real)')
plt.plot(range(len(train), len(train)+horizonte), previsoes, marker='o', label='Previsão AR(p)')
plt.legend()
plt.tight_layout()
plt.show()
```

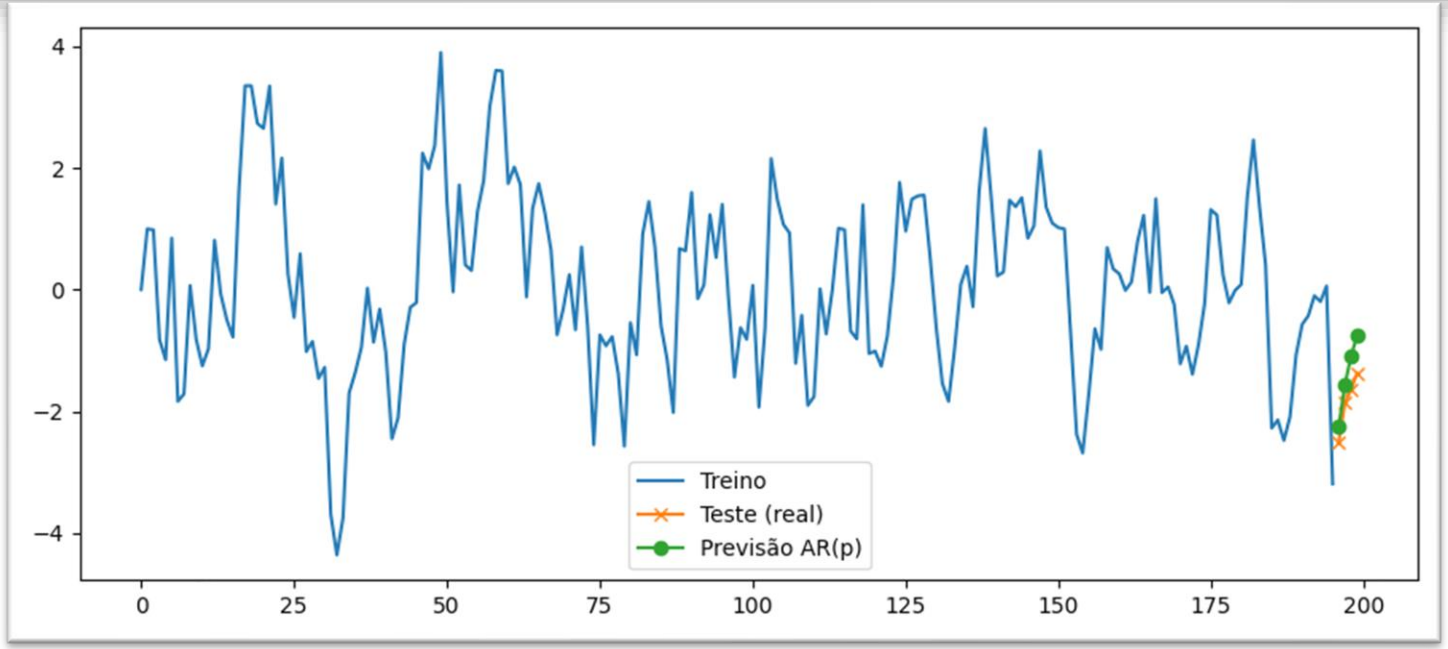


Gráfico e próximos passos

Para finalizar realize o Ljung-Box e discuta a performance do modelo!

Desafio

Desafio:

Agora considere as séries estacionárias que você enviou na atividade passada e realize a implementação do modelo AR(p) para one-step-ahead e multi-step-ahead. Lembre-se de passar por todo o Pipeline.

Instituto J&F

Centro de Educação para os Negócios